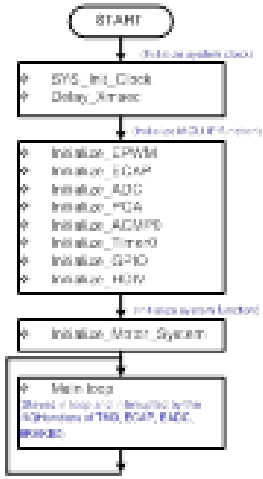
Introduction to Basic Motor Control System by NM1240

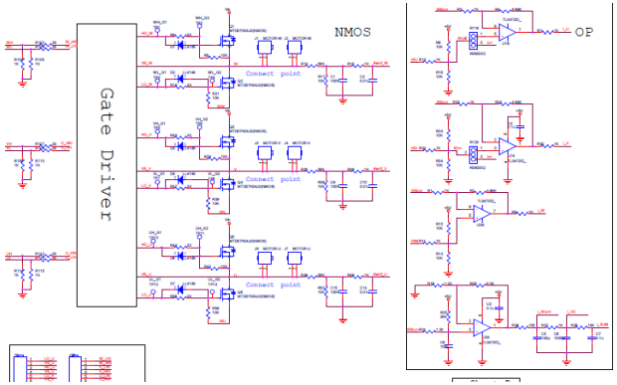
2021/May

Demo Purpose

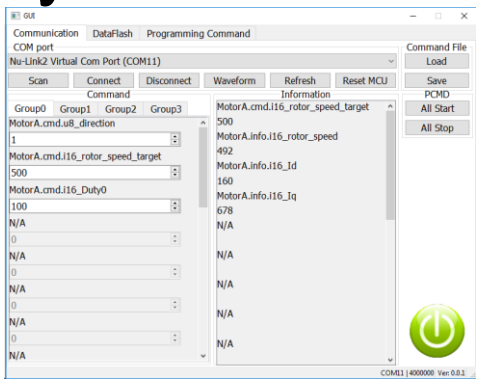
- **Easily & Rapidly Getting Start for Motor Application**
  - Provide demo system for refer-design

- **Motor Drive System**
  - Elementary S/W flow
  - Refer H/W circuit



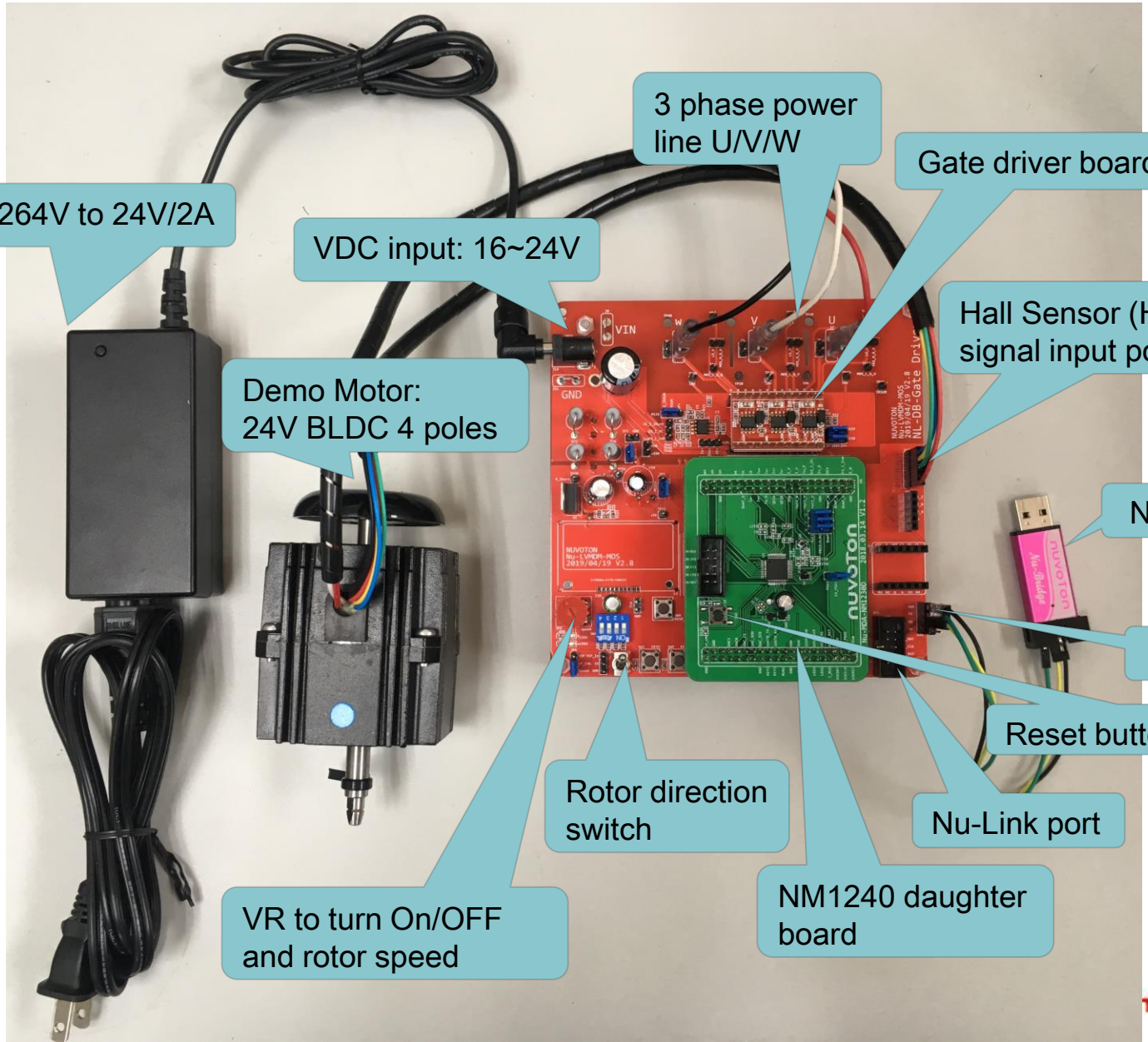


- **Friendly GUI Interface Linking PC and System Board through UART(COM port)**
  - User can easily demo motor behavior



nuvoTon confidential

Demo System



ntial

Overview

- Demo how to use NM1240 to implement a basic motor drive system with
  - Hall + FOC + 2-shunt R
  - Hall + FOC + 1-shunt R
  - Hall + Six step square wave control
    - Duty open loop
    - Speed close loop
- Customer may refer this demo system to develop own specific system.

Demo Function

- ✓ **Demo NM1240 MCU basic setting**
  - Initialize MCU
  - Timer
  - Interrupt
  - ECAP (Enhance Capture Timer/Counter)
  - Complementary PWM0~5 setting
  - 12-bit 2-S/H ADC
  - OP (Operational Amplifier)
  - ACMP (Analog Comparator)
  - USCI (UART)
  - GDMA
- ✓ **Basic PI control** (Proportional and Integral control) for speed and current control
- ✓ **Basic FOC + SVPWM**

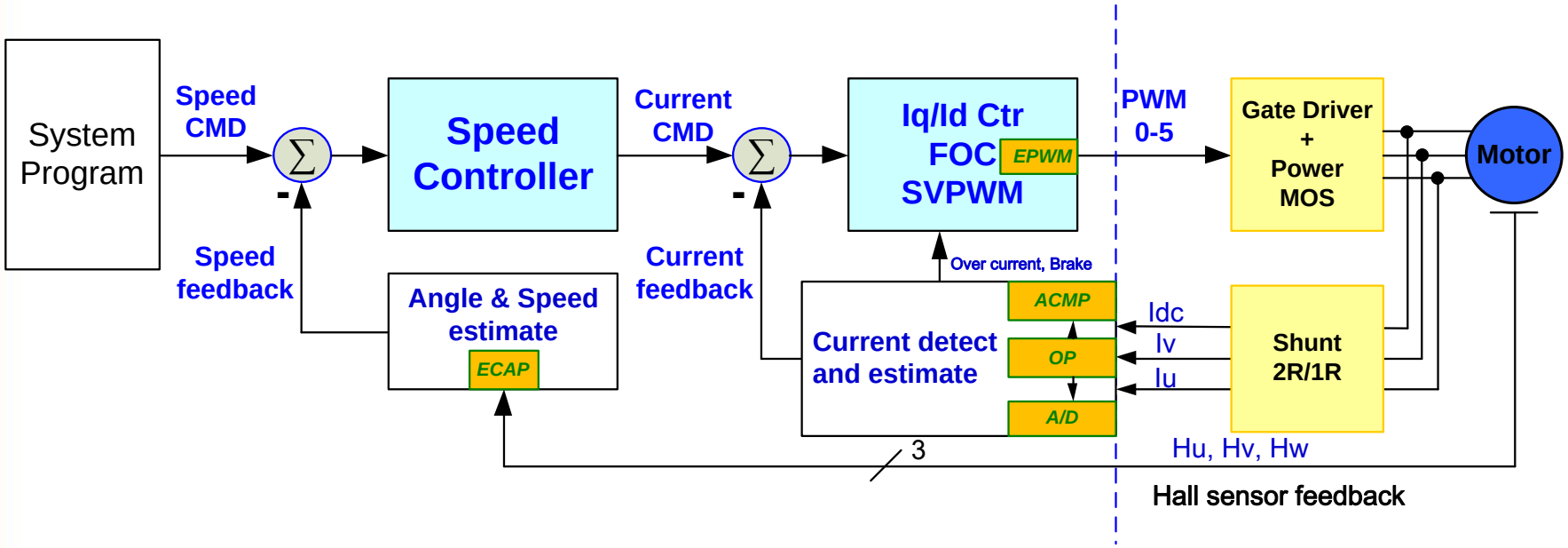
nuvoTon confidential

Demo Function

- ✓ 6-Step square wave control
- ✓ Basic motor speed estimation by ECAP
- ✓ System control mode
  - VSP control mode : External VR to turn on/off the motor and adjust speed
  - UART control mode : Set command from PC to MCU through UART protocol.

Speed Control by FOC

- Demo System : Hall + FOC
- Control mode : 2R / 1R (R : current shunt resistance)

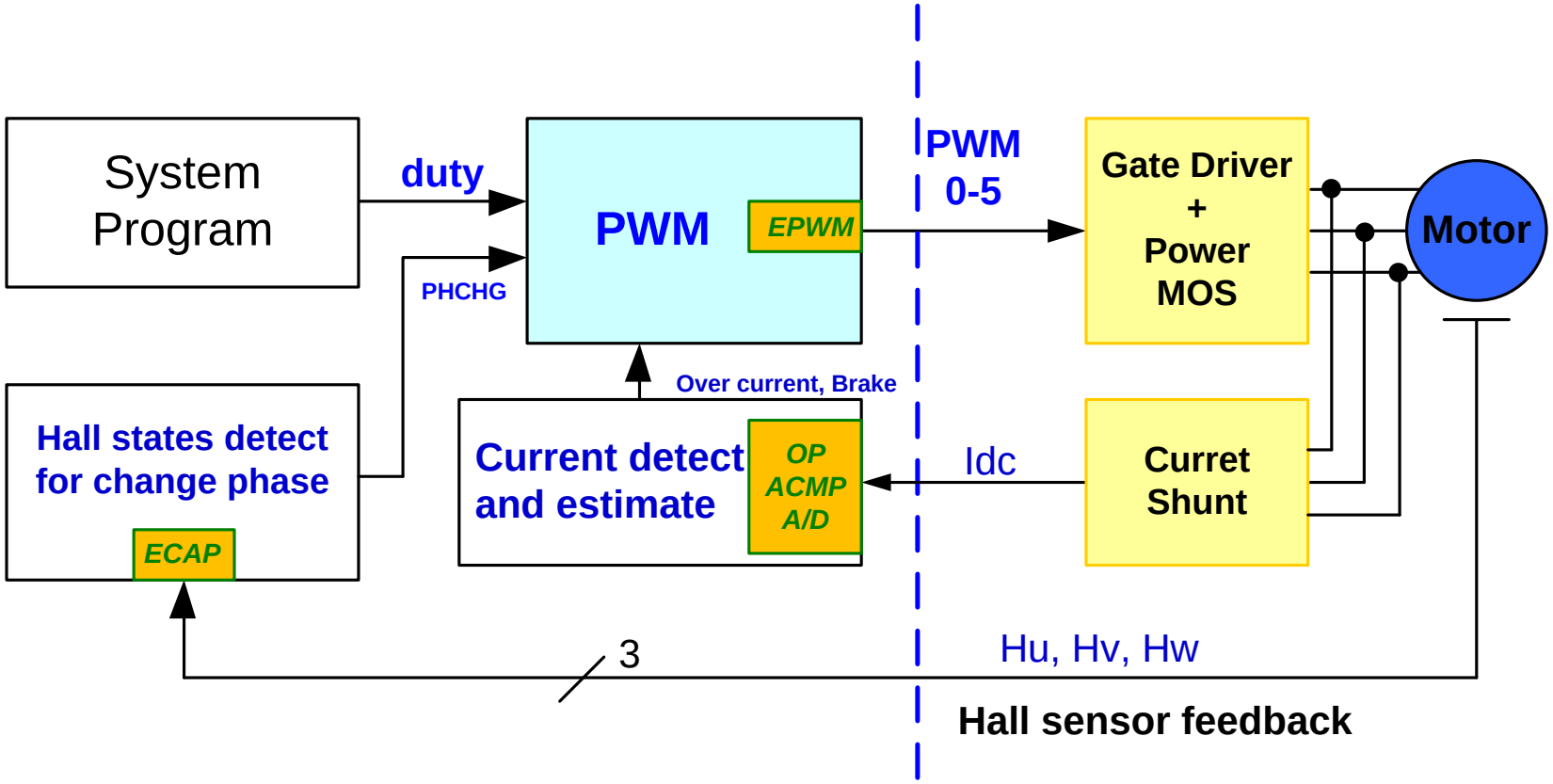


Demo System
Hall + FOC + 2R

nuvoTon confidential

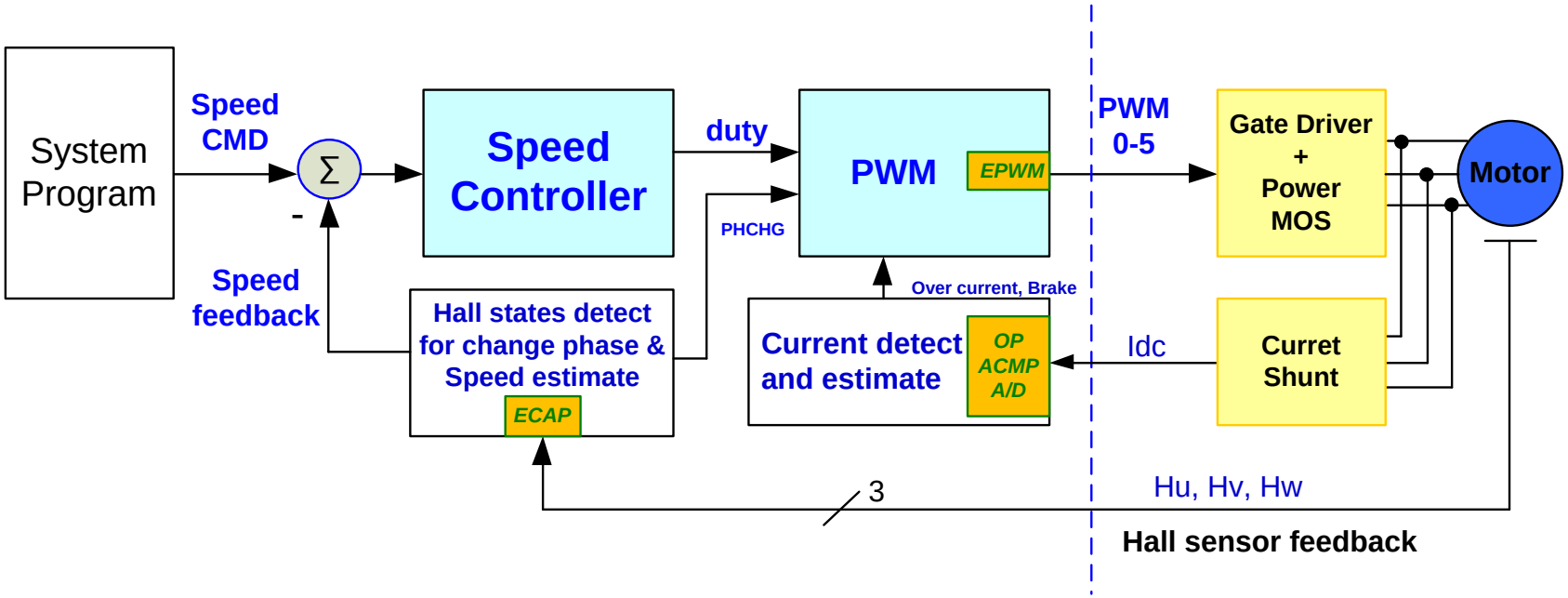
PWM Duty Control by 6-Step

- Demo System : Hall + Six Step square wave control
- Control mode : Duty open loop



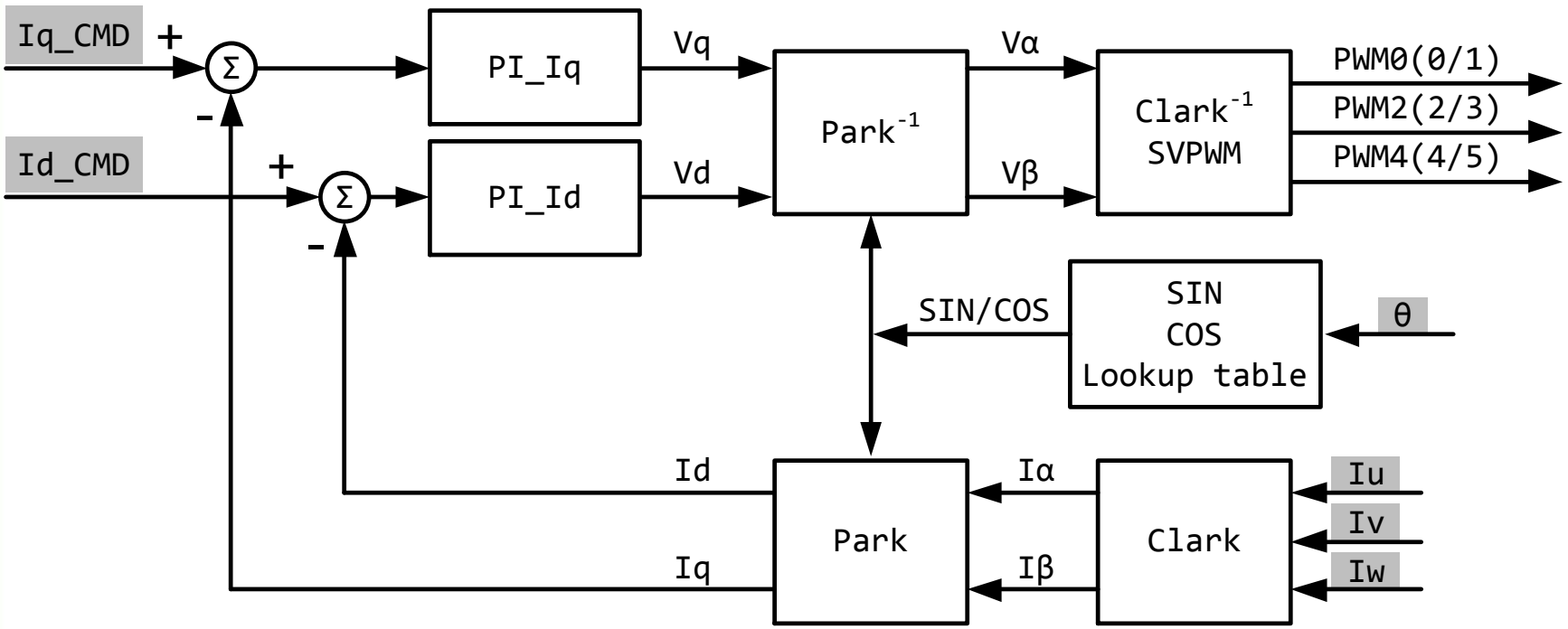
Speed Control by 6-Step

- Demo System : Hall + Six Step square wave control
- Control mode : Speed close loop



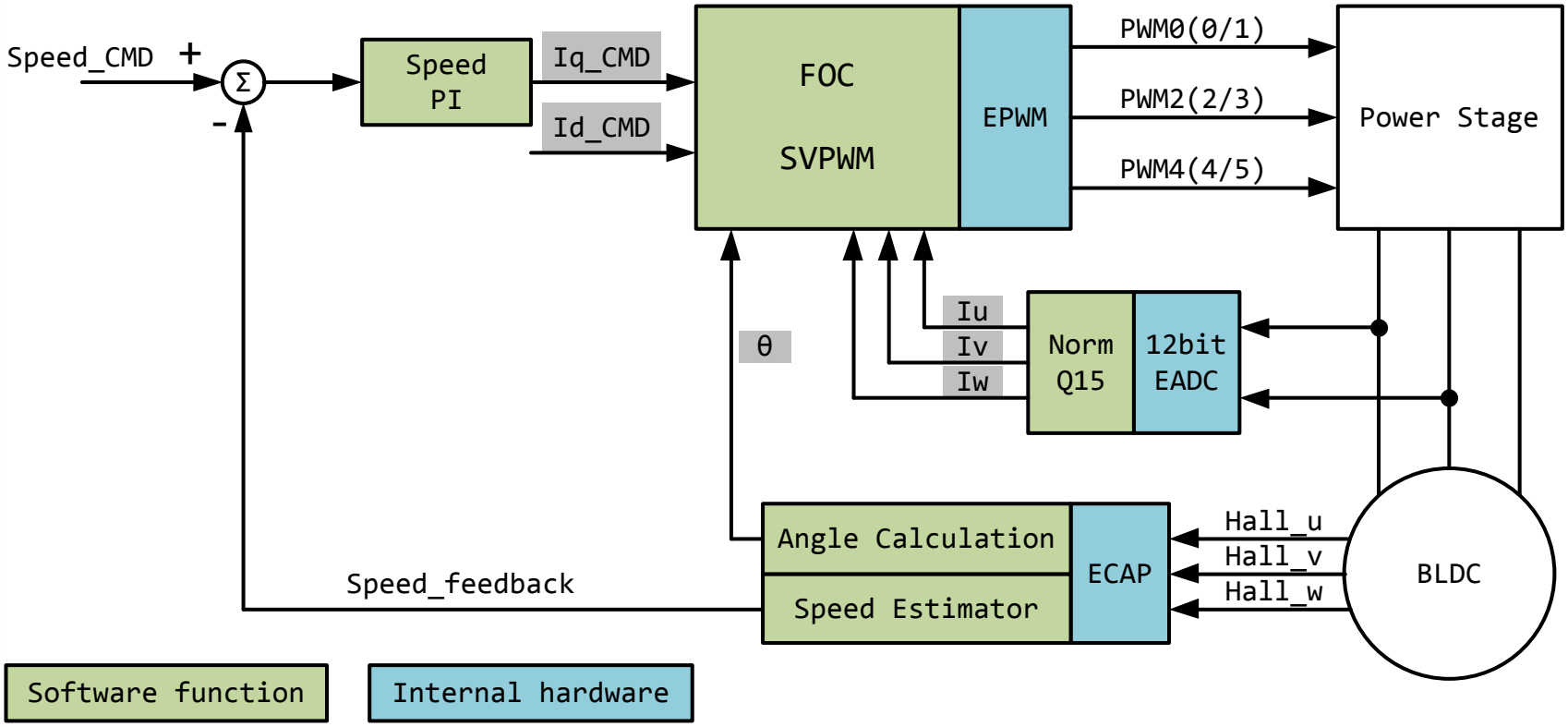
FOC by Software Library

- FOC: Field Oriented Control, 磁場導向控制
- Vector Control: 向量控制, 矢量控制
- a-b-c stationary frame
- α-β stationary frame
- d-q rotating frame



nuvoTon confidential

FOC control block



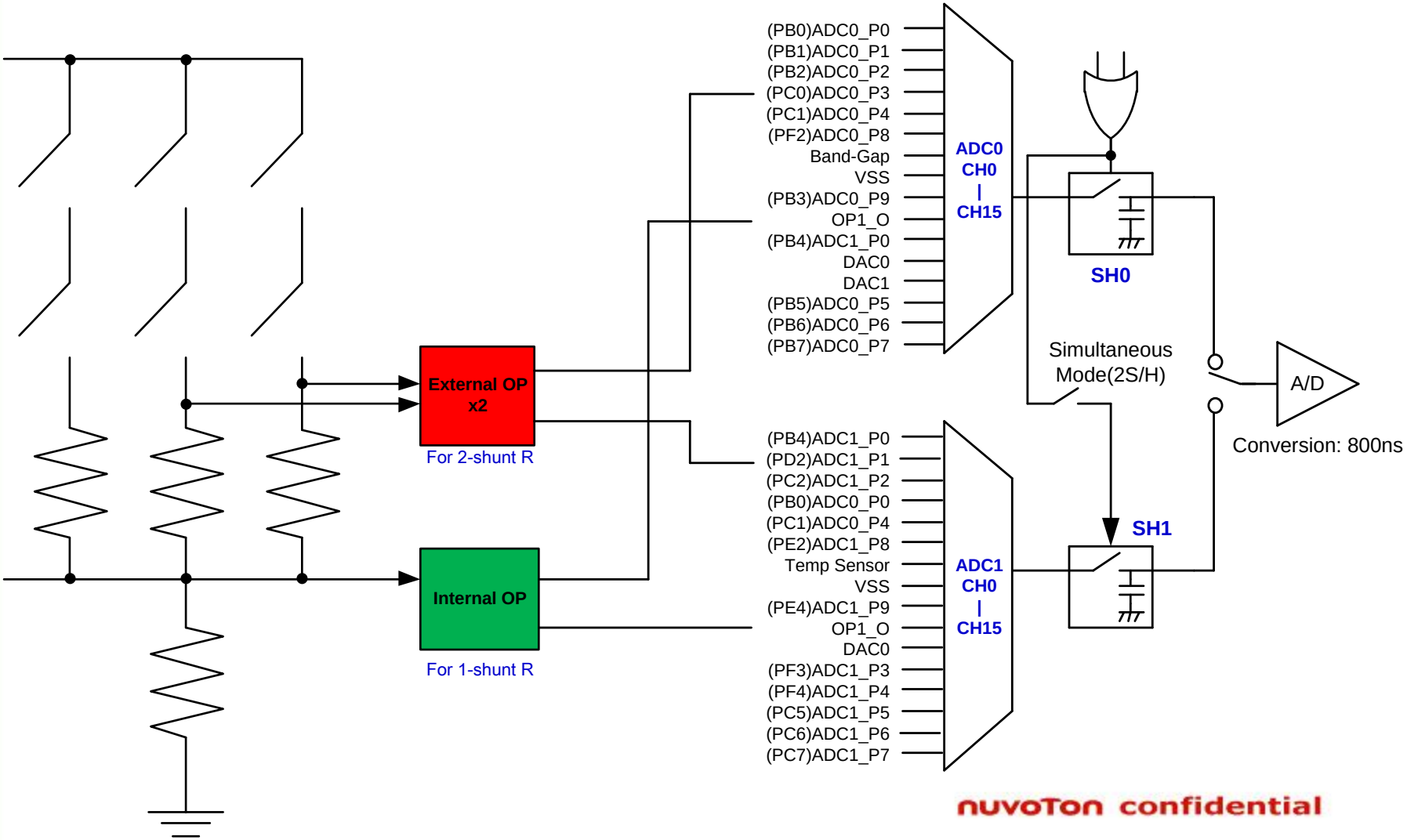
FOC Function Call

- void **Iuvw_to_Idq**(AMotor* Motor, int32 sin, int32 cos)
  - Do Clark Transfer: Transfer Iu/v/w to Ialfa/beta
  - Do Park Transfer: Transfer Ialfa/beta to Id/q
- void **Vdq_to_SVPWM**(AMotor* Motor, EPWM_T* epwm, int32 pwm_full_scale, int32 pwm_max_duty, int32 sin, int32 cos)
  - Do Inv_Park Transfer: Transfer Vd/q to Valfa/beta
  - Do Modified Inv_Clark Transfer: Transfer Valfa/Vbeta to Vrefx/y/z
  - Do SVPWM calculation to produce Duty0/2/4
  - Update the duty to EPWM->CMPDAT[0/2/4]

nuvoTon confidential

Phase Current Detection by ADC

- 2-shunt R : Iu / Iv through internal or external two OP
- 1-shunt R : Iu / Iv / Iw through internal OP



nuvoTon confidential

Six Step square wave control

- Sequentially output PWM according with Hall-State.
- Do phase change by setting EPWM Phase Change Register (EPWM_PHCHG).

Forward

STAGE FW	HALL STATES	HW PB2	HV PB1	HU PB0	Exciting phase	Gate signal					
						U_H	U_L	V_H	V_L	W_H	W_L
1	5	1	0	1	u→w	PWM	0	0	0	0	1
2	1	0	0	1	v→w	0	0	PWM	0	0	1
3	3	0	1	1	v→u	0	1	PWM	0	0	0
4	2	0	1	0	w→u	0	1	0	0	PWM	0
5	6	1	1	0	w→v	0	0	0	1	PWM	0
6	4	1	0	0	u→v	PWM	0	0	1	0	0

Reverse

STAGE RW	HALL STATES	HW PB2	HV PB1	HU PB0	Exciting phase	Gate signal					
						U_H	U_L	V_H	V_L	W_H	W_L
1	3	0	1	1	u→v	PWM	0	0	1	0	0
2	1	0	0	1	w→v	0	0	0	1	PWM	0
3	5	1	0	1	w→u	0	1	0	0	PWM	0
4	4	1	0	0	v→u	0	1	PWM	0	0	0
5	6	1	1	0	v→w	0	0	PWM	0	0	1
6	2	0	1	0	u→w	PWM	0	0	0	0	1

nuvoTon confidential

Six Step square wave Function Call

- void SIX_STEP_CHANGE_PHASE_FW(void)
  - Do phase change function for rotor forward
  - Using EPWM Phase Change Register (EPWM_PHCHG) to change phase

- void SIX_STEP_CHANGE_PHASE_RW(void)
  - Do phase change function for rotor reverse
  - Using EPWM Phase Change Register (EPWM_PHCHG) to change phase

nuvoTon confidential

Motor Variable Structure

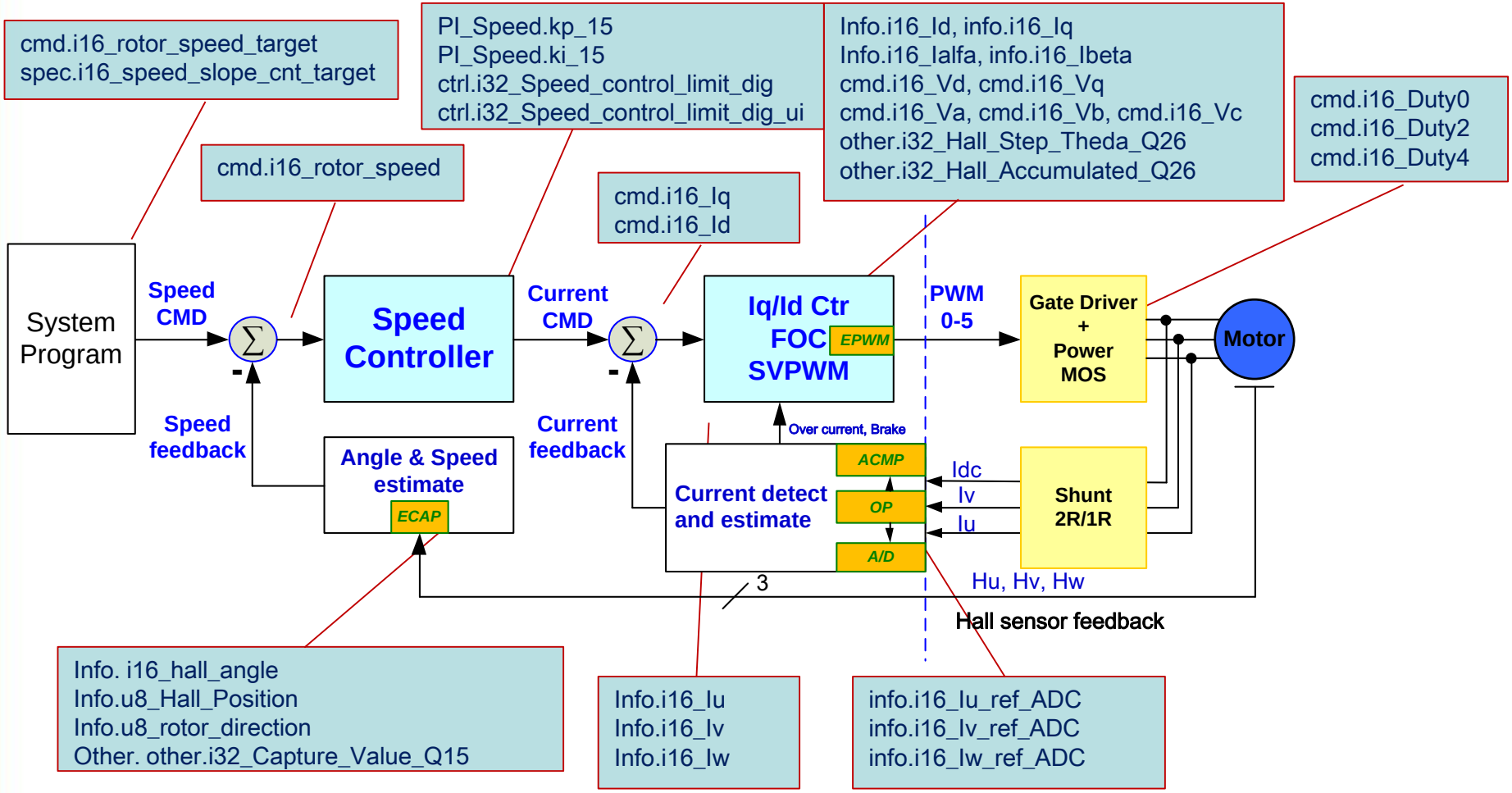
```
typedef struct tag_Motor
 {
     AMotorSpec          spec;    馬達系統基本規格, ex: PWM 頻率
     AMotorInfo          info;    即時的data, ex: Iu, Iv, Iw, rotor_speed
     AMotorCommand       cmd;     馬達驅動的命令, ex:轉子速度命令rpm
     AMotorController    ctrl;    控制器的最大限制值
     AMotorSym           sym;     系統資訊, ex: 過流錯誤回報資料
     AMotorOther         other;   一些系統程式執行時需要的變數或flag
 } AMotor;
```

nuvoTon confidential

PI Controller Variable Structure

```
typedef struct
 {
        INT32   e1;             Error = cmd – info, Q15 format
        INT32   ui_31;          Accumulated integrator, Q31 format
        INT32   kp_31,ki_31;    Parameters of Kp & Ki, Q31 format
 }PICs;
```

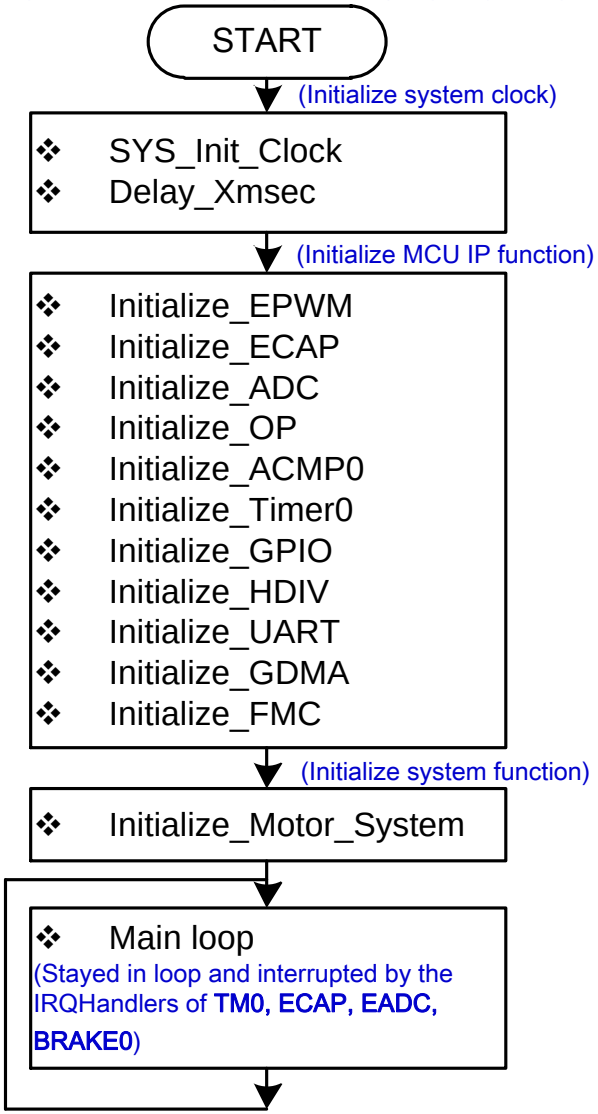
Key Variables



nuvoTon confidential

Flow Chart of main.c

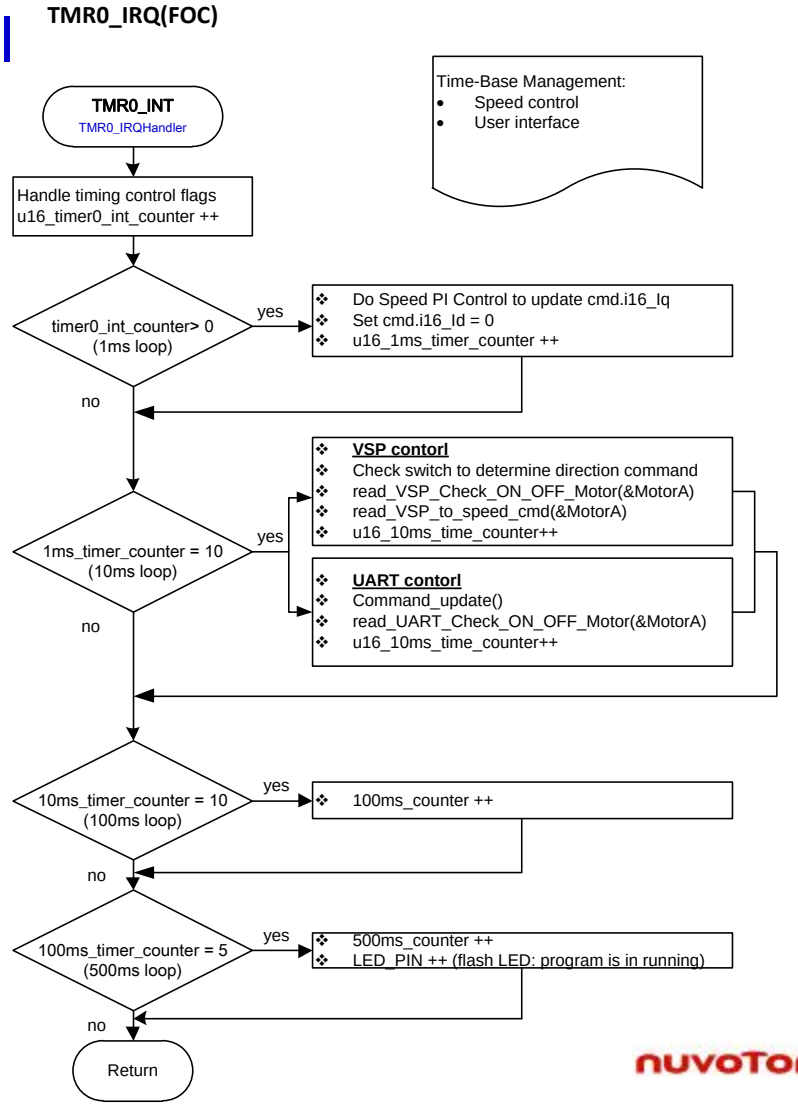
- Main program to initialize MCU and system



nuvoTon confidential

Flow Chart of TMR0_INT (for FOC)

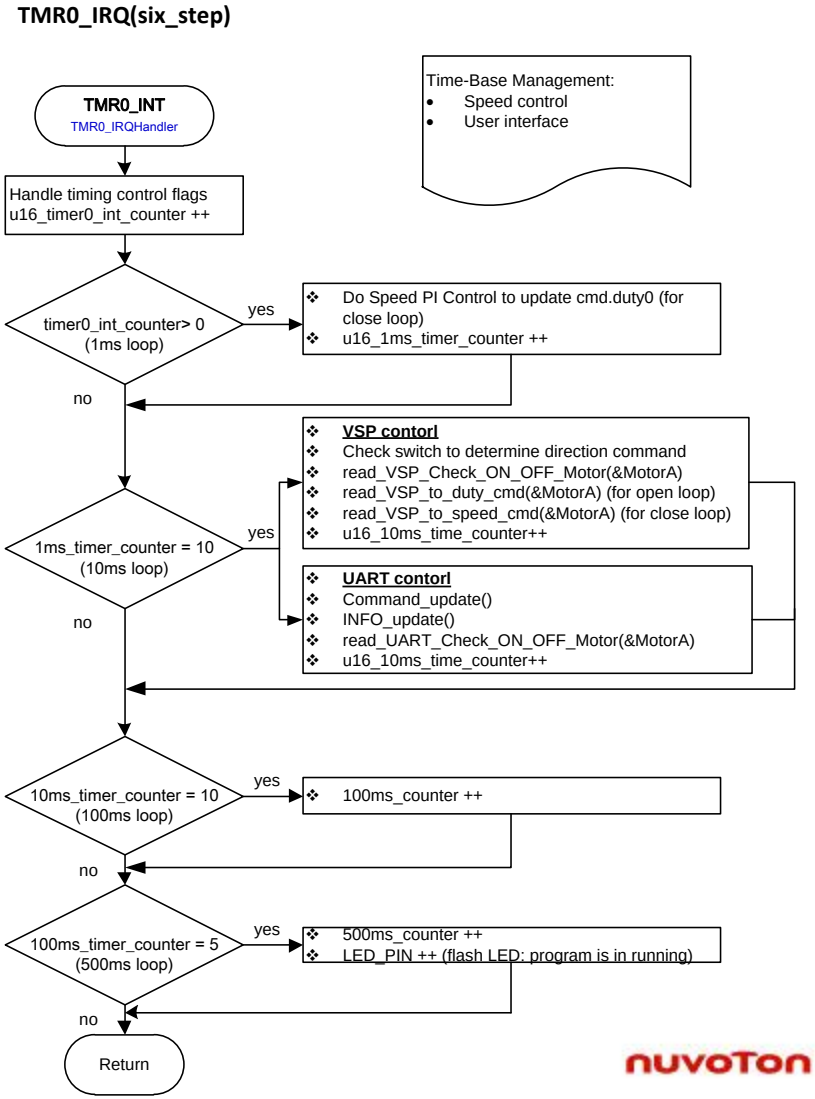
- Timer0 INT for system timer handles the user interface and speed control



nuvoTon confidential

Flow Chart of TMR0_INT (for Six step)

- Timer0 INT for system timer handles the user interface and speed control



nuvoTon confidential

Flow Chart of EADC_INT (for 2R)

- EADC INT: for current control and FOC

EADC_INT:
EADC interrupt which is periodically triggered by PWM every 66.6us handles the current detection, control and FOC.

EADC_INT
EADC0_IRQHandler

- Re-construct 3 phase current info.i16_Iu/Iv/Iw
- Update speed command according with speed acceleration slope: **cmd_speed_slope_with_hall()**
- Clark and Park transform: **Iuvw_to_Idq()**
- Current control: **PI_Id_current()** & **PI_Iq_current()**
- Inv(Park), Inv(Clark) and SVPWM: **Vdq_to_SVPWM()**
- Update hall angle: info.i16_hall_angle
- ADC read VSP and Idc from OP: **SW_Get_Speed_Command_From_VR(), SW_Get_I_BUS_From_Ishunt()**

Return

nuvoTon confidential

Flow Chart of EPWM_INT (for 1R)

- EPWM INT: for current control and FOC (1R)

EPWM_INT:
EPWM interrupt is periodically triggered by PWM every 66.6us. It will call PWM_Period_INT_1R() to handle the current detection, control and FOC.

EPWM_INT
EPWM_IRQHandler

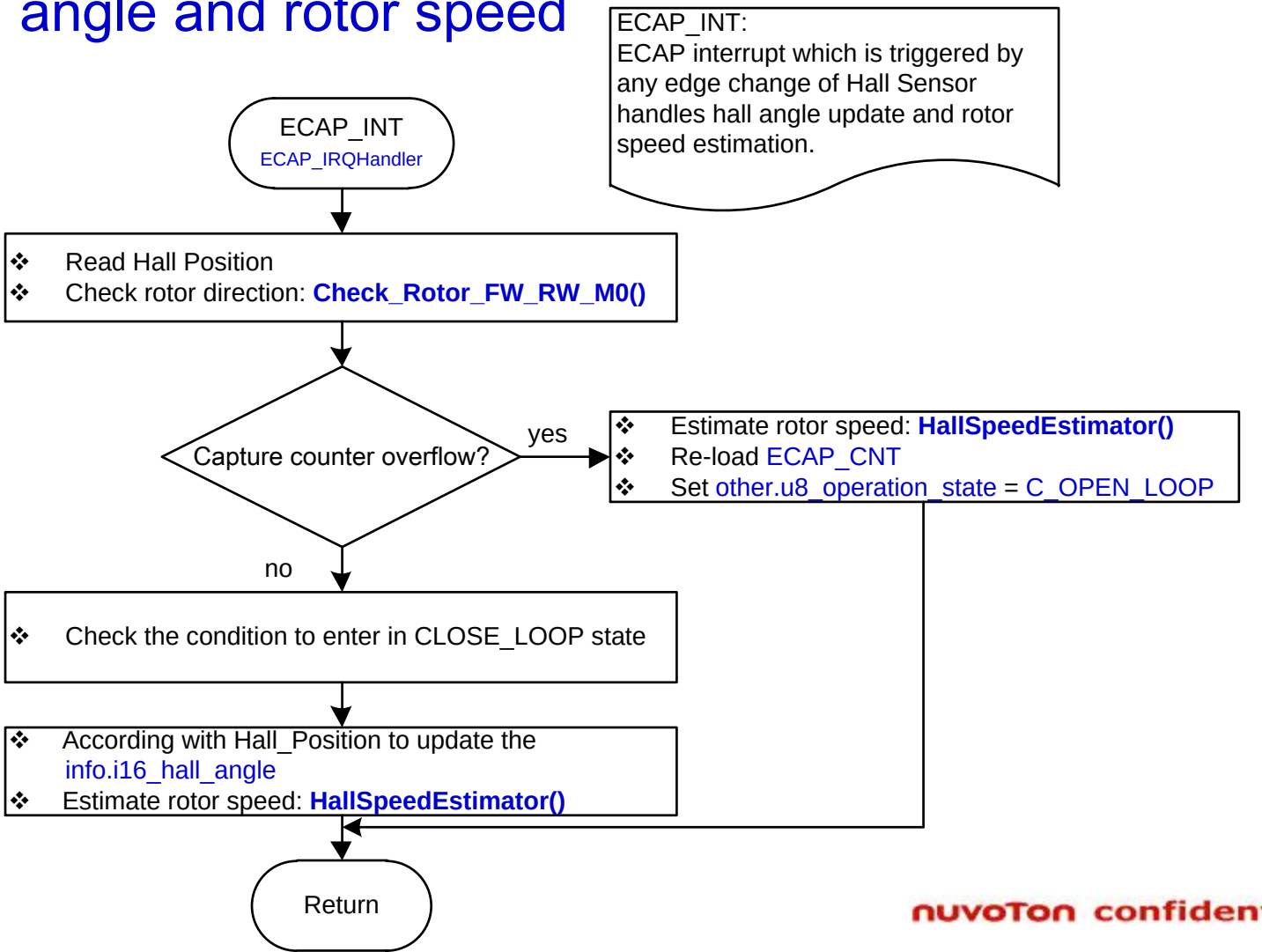
- Call **PWM_Period_INT_1R()** to do the following task
- Store last two measurement of phase current
- ADC read VSP:
  SW_Get_Speed_Command_From_VR()
- Set ADC is HW triggered by PWM falling edge
  Set_HW_ADC_Read_OP1_O_1R_by_Indpt_Mode()
- Re-construct 3 phase current info.i16_Iu/Iv/Iw
- Update speed command according with speed acceleration slope: **cmd_speed_slope_with_hall()**
- Clark and Park transform: **Iuvw_to_Idq()**
- Current control: **PI_Id_current()** & **PI_Iq_current()**
- Inv(Park), Inv(Clark) and SVPWM: **Vdq_to_SVPWM()**
- Update hall angle: info.i16_hall_angle

Return

nuvoTon confidential

Flow Chart of ECAP_INT(for FOC)

- ECAP INT for capturing hall sensor signals to update the hall angle and rotor speed

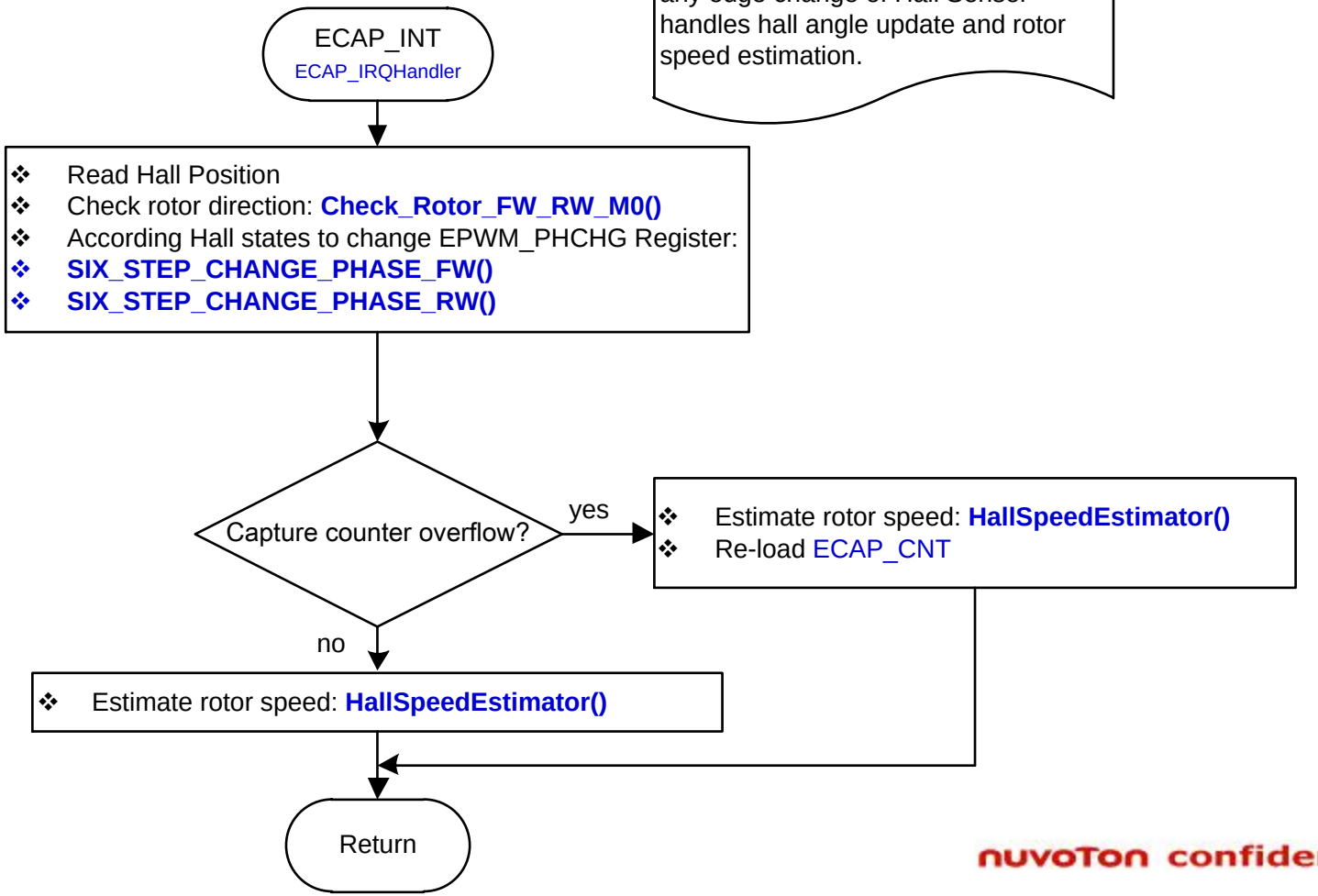


nuvoTon confidential

Flow Chart of ECAP_INT(for Six step)

- ECAP INT for capturing hall sensor signals to update the hall angle and rotor speed

ECAP_INT:
ECAP interrupt which is triggered by any edge change of Hall Sensor handles hall angle update and rotor speed estimation.



nuvoTon confidential

Flow Chart of BRAKE0_INT

- BRAKE0 INT to STOP PWM and turn off MOS when over current

BRAKE0_INT:
BRAKE0 interrupt is triggered by the rising-edge change of ACMP0.
It forces the EPWM to turn off all power MOS and re-start PWM counter.

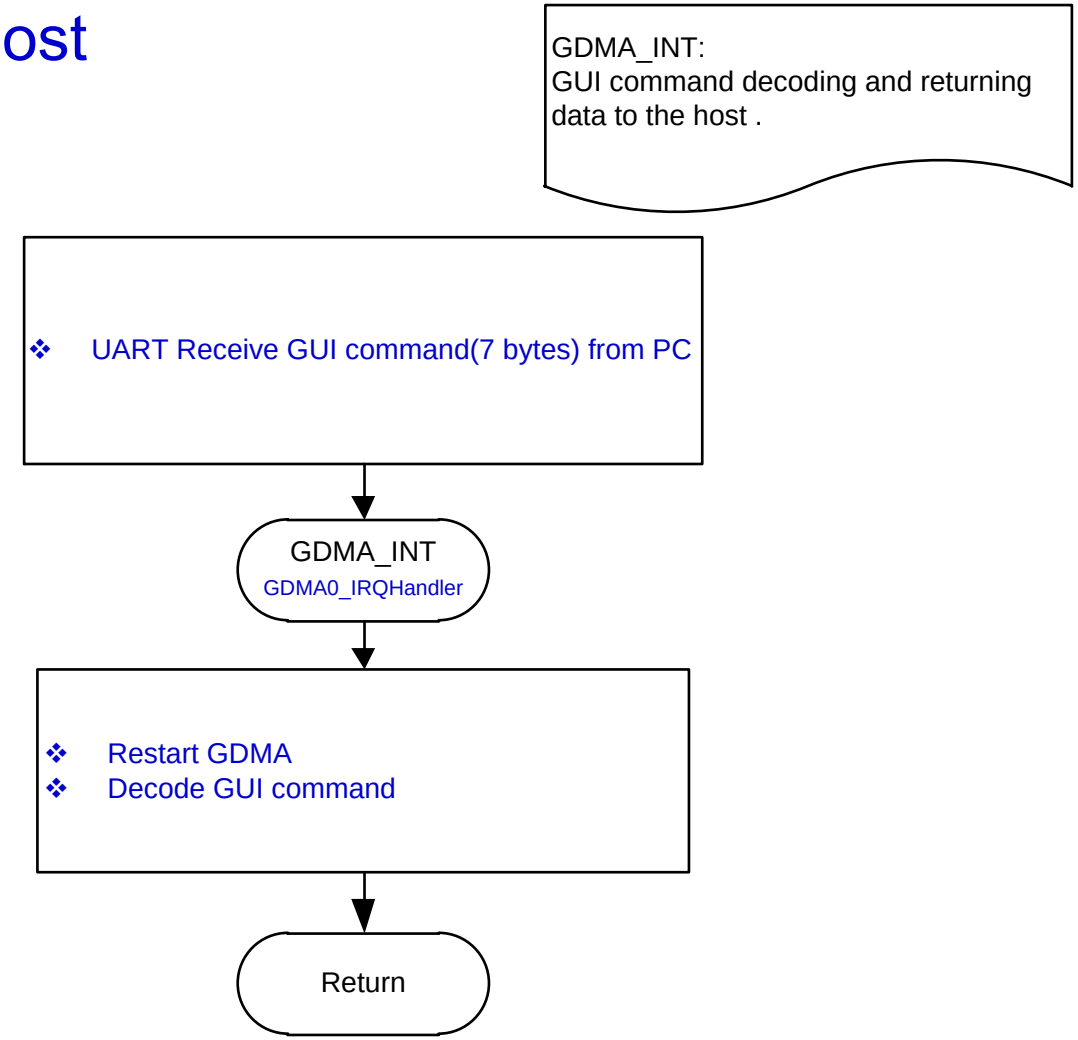
BRAKE0_INT
BRAKE0_IRQHandler

- Hardware set PWM[5:0] = BKOD[5:0] (preset in initial procedure)
- Set EPWM to turn off power MOS: **Set_EPWM_MASK_Enable(PWM_OUT_OFF_MOS)**
- Re-start EPWM counter. (EPWM stops counting when Brake0 is asserted)

Return

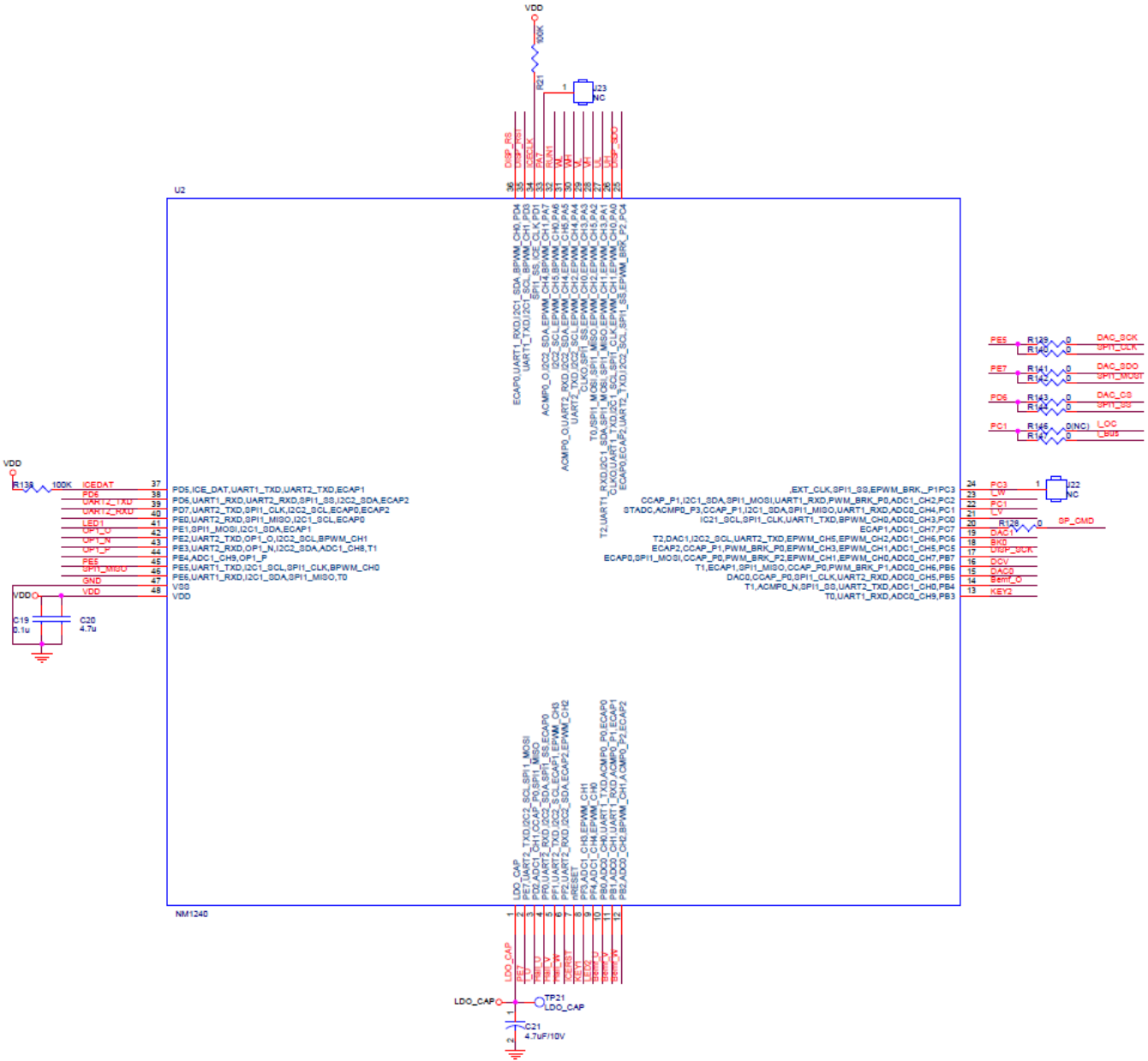
Flow Chart of UART_INT

- GDMA INT for GUI command decoding and returning data to the host



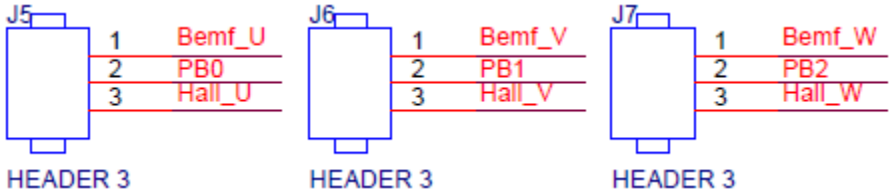
nuvoTon confidential

Pin Configure vs Circuit (I)



Pin Configure vs Circuit (II)

- ✓ PWM output (6 pin) by EPWM
  - (PWM5~PWM0) = (PA5~PA0)
- ✓ Direction command from RUN switch (1 pin) by GPIO
  - PA6: read the state of RUN switch
- ✓ Light LED (1 pin) by GPIO
  - PF4: control LED on/off
- ✓ Amplifier by internal OP
  - PE2, PE3, PE4: OP1_O, OP1_N, OP1_P



- ✓ Hall sensor (3 pins) by ECAP
  - (Hw, Hv, Hu) = (PF2, PF1, PF0) = (ECAP2, ECAP1, ECAP0)

- ✓ UART port (2 pin) by USCI-UART0
  - UART2_RXD = PE0
  - UART2_TXD = PD7
- ✓ Nu-Link ICE port by SWD
  - ICE_DAT = PD5
  - ICE_CLK = PD1
- ✓ (Optional) DAC module by SPI0
  - SPI1_MOSI = PE7
  - SPI1_CLK = PE5
  - CS = PD6 (GPIO mode)

nuvoTon confidential

Pin Configure in Demo System (I)

- ✓ **Hall sensor (3 pins) by ECAP**
  - (Hw, Hv, Hu) = (PF2, PF1, PF0) = (ECAP2, ECAP1, ECAP0)
- ✓ **Phase current feedback (2 pins) by ADC** (for 2-shunt R)
  - Iu: ADC1_P1 = PD2
  - Iv: ADC0_P3 = PC0
- ✓ **Total current amplify (2 pin) by OP**
  - IRSum = OP1_P = PE4
  - Set OP gain = 5
  - OP1_O = PE2 →RC filter → I_OC = ACMP0_P3
  - Internally OP1_O to ADC for phase current measurement (1-shunt R)
- ✓ **Over current detect (1 pin) by ACMP0**
  - I_OC = ACMP0_P3 = PC1
- ✓ **Speed command from VR (1 pin) by ADC**
  - Speed = ADC1_P7 = PC7

nuvoTon confidential

Pin Configure in Demo System (II)

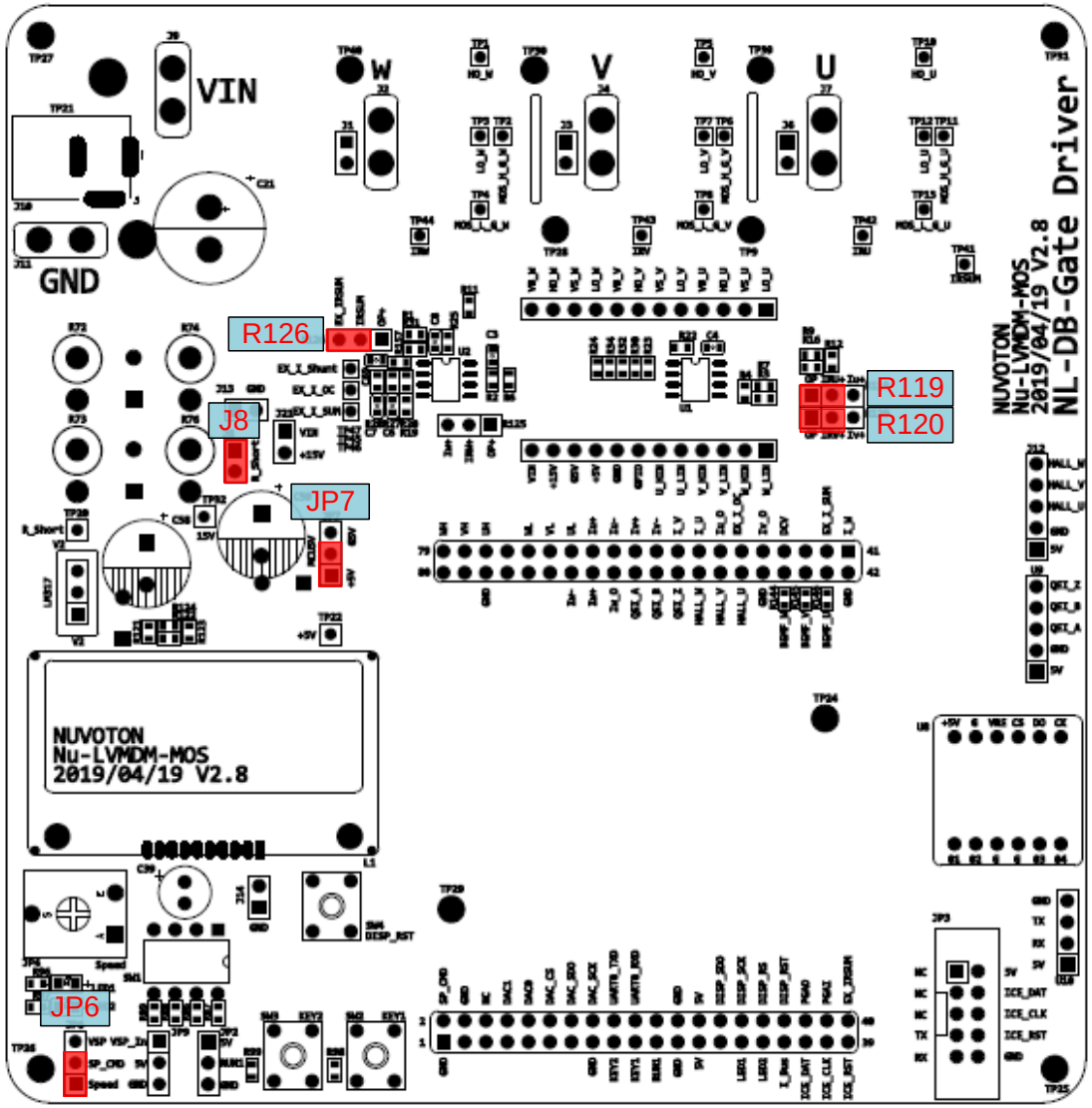
- ✓ PWM output (6 pin) by EPWM
  - (PWM5~PWM0) = (PA5~PA0)
- ✓ Direction command from RUN switch (1 pin) by GPIO
  - PA6: read the state of RUN switch
- ✓ Light LED (1 pin) by GPIO
  - PF4: control LED on/off
- ✓ UART port (2 pin) by USCI-UART2
  - UART2_RXD = PD7
  - UART2_TXD = PE0
- ✓ Nu-Link ICE port by SWD
  - ICE_DAT = PD5
  - ICE_CLK = PD1

nuvoTon confidential

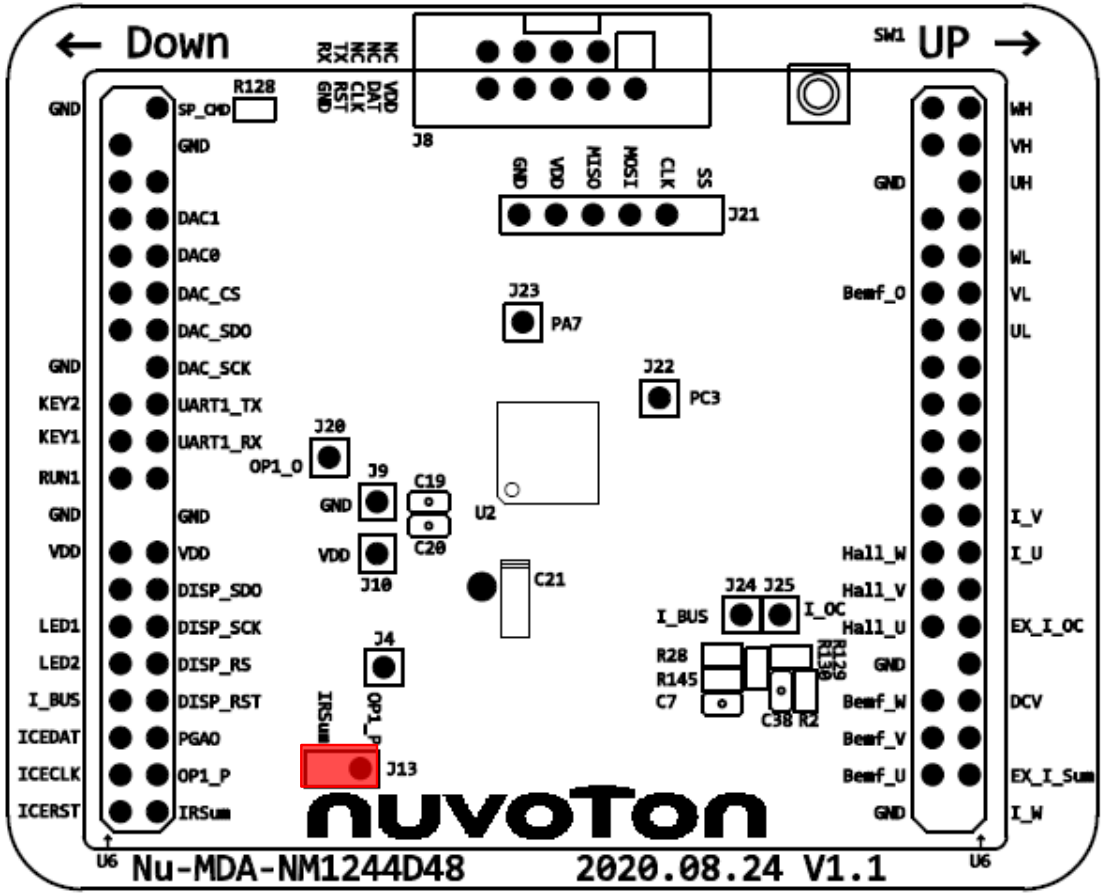
Pin Configure in Demo System (III)

- ✓ Amplifier by internal OP
  - OP1_O = PE2
  - OP1_N = PE3
  - OP1_P = PE4
- ✓ (Optional) DAC module by SPI1
  - SPI1_MOSI = PE7
  - SPI1_CLK = PE5
  - CS = PD6 (GPIO mode)

Demo board jumper layout (I)



Demo board jumper layout (II)



Pin Configure in Demo System (IV)

GPIO	Pin	GPIO MFP0	Function ICE XTAL MFP1	ADC MFP2	PWM_BRAKE MFP3	ACMP0 MFP4	ACMP1 MFP5	OP MFP6	TIMER MFP7	I2C MFP8	SPI1 MFPA	UART MFPB	ECAP MFPC	MFPE	MFPF	Remark
GPA0	26	GPIO	CLKO		***EPWM_CH0***					I2C1_SCL	SPI1_CLK	UART1_TXD			EPWM_CH1	UH
GPA1	27	GPIO			***EPWM_CH1***				T2	I2C1_SDA	SPI1_MOSI	UART1_RXD		SPI1_MISO	EPWM_CH3	UL
GPA2	28	GPIO			***EPWM_CH2***				T0		SPI1_MISO			SPI1_MOSI	EPWM_CH5	VH
GPA3	29	GPIO	CLKO		***EPWM_CH3***						SPI1_SS				EPWM_CH0	VL
GPA4	30	GPIO			***EPWM_CH4***					I2C2_SCL		UART2_TXD			EPWM_CH2	WH
GPA5	31	GPIO			***EPWM_CH5***	ACMP0_O				I2C2_SDA		UART2_RXD			EPWM_CH4	WL
GPA6	32	***GPIO***			EPWM_CH5									I2C2_SCL	BPWM_CH0	RUN
GPA7	33	***GPIO***			EPWM_CH4	ACMP0_O								I2C2_SDA	BPWM_CH1	
GPB0	10	GPIO		ADC0_P0		ACMP0_P0			ECAP0			UART1_TXD	ECAP0			BemfU
GPB1	11	GPIO		ADC0_P1		ACMP0_P1			ECAP1			UART1_RXD	ECAP1			BemfV
GPB2	12	GPIO		ADC0_P2	BPWM_CH1	ACMP0_P2			ECAP2			UART1_TXD	ECAP2	T2		BemfW
GPB3	13	***GPIO***		ADC0_P9					T0			UART1_RXD				KEY2
GPB4	14	GPIO		ADC1_P0		ACMP0_N			T1		SPI1_SS	UART2_TXD				Bemf_O
GPB5	15	GPIO		ADC0_P5		***DAC0***			CCAP_P0		SPI1_CLK	UART2_RXD				DAC 0
GPB6	16	***GPIO***		ADC0_P6	PWM_BRK_P1				CCAP_P0		SPI1_MISO		ECAP1	T1		DCV
GPB7	17	***GPIO***		ADC0_P7	PWM_BRK_P2				CCAP_P0		SPI1_MOSI		ECAP0	EPWM_CH0	EPWM_CH1	DISP_SCK
GPC0	21	GPIO		***ADC0_P3***	BPWM_CH0					I2C1_SCL	SPI1_CLK	UART1_TXD				I_V
GPC1	22	GPIO		***ADC0_P4***	STADC	ACMP0_P3			CCAP_P1	I2C1_SDA	SPI1_MISO	UART1_RXD				I_Bus
GPC2	23	GPIO		***ADC1_P2***	PWM_BRK_P0				CCAP_P1	I2C1_SDA	SPI1_MOSI	UART1_RXD				I_W
GPC3	24	GPIO	REF_CLK		PWM_BRK_P1						SPI1_SS					EXT_CLK
GPC4	25	***GPIO***			PWM_BRK_P2				ECAP0	I2C2_SCL	SPI1_SS	UART2_TXD	ECAP2			DISP_SDO
GPC5	18	GPIO		ADC1_P5	***PWM_BRK_P0***				CCAP_P1				ECAP2	EPWM_CH1	EPWM_CH3	外部ACMP_O，forBRK
GPC6	19	GPIO		ADC1_P6	EPWM_CH2		***DAC1***		T2	I2C2_SCL		UART2_TXD			EPWM_CH5	DAC 1
GPC7	20	GPIO		***ADC1_P7***									ECAP1			SP_SMD
GPD1	34	GPIO	***ICE_CLK***								SPI1_SS					ICE CLK
GPD2	3	GPIO		***ADC1_P1***					CCAP_P0		SPI1_MISO					I_U
GPD3	35	***GPIO***			BPWM_CH1					I2C1_SCL		UART1_TXD				DISP_RST
GPD4	36	***GPIO***			BPWM_CH0					I2C1_SDA		UART1_RXD	ECAP0			DISP_RS
GPD5	37	GPIO	***ICE_DAT***										ECAP1	UART1_TXD	UART2_TXD	ICE DAT
GPD6	38	GPIO								I2C2_SDA	***SPI1_SS***		ECAP2	UART1_RXD	UART2_RXD	DAC-CS
GPD7	39	GPIO							ECAP0	I2C2_SCL	SPI1_CLK	***UART2_TXD***	ECAP2			UART_TX
GPE0	40	GPIO								I2C1_SCL	SPI1_MOSI	***UART2_RXD***	ECAP0			UART_RX
GPE1	41	***GPIO***								I2C1_SDA	SPI1_MISO		ECAP1			LED1
GPE2	42	GPIO			BPWM_CH1			***OP1_O***		I2C2_SCL		UART2_TXD				OP1 O
GPE3	43	GPIO		ADC1_P8				***OP1_N***	T1	I2C2_SDA		UART2_RXD				OP1 N
GPE4	44	GPIO		ADC1_P9				***OP1_P***								OP1 P
GPE5	45	GPIO			BPWM_CH0					I2C1_SCL	***SPI1_CLK***	UART1_TXD				DAC-SCK
GPE6	46	GPIO							T0	I2C1_SDA	SPI1_MISO	UART1_RXD				
GPE7	2	GPIO								I2C2_SCL	***SPI1_MOSI***	UART2_TXD				DAC-SDO
GPF0	4	GPIO								I2C2_SDA	SPI1_SS	UART2_RXD	***ECAP0***			Hall_U
GPF1	5	GPIO			EPWM_CH3					I2C2_SCL		UART2_TXD	***ECAP1***			Hall_V
GPF2	6	GPIO		ADC0_P8	EPWM_CH2					I2C2_SDA		UART2_RXD	***ECAP2***			Hall_W
GPF3	8	***GPIO***		ADC1_P3	EPWM_CH1											KEY1
GPF4	9	***GPIO***		ADC1_P4	EPWM_CH0											LED2

nuvoTon confidential

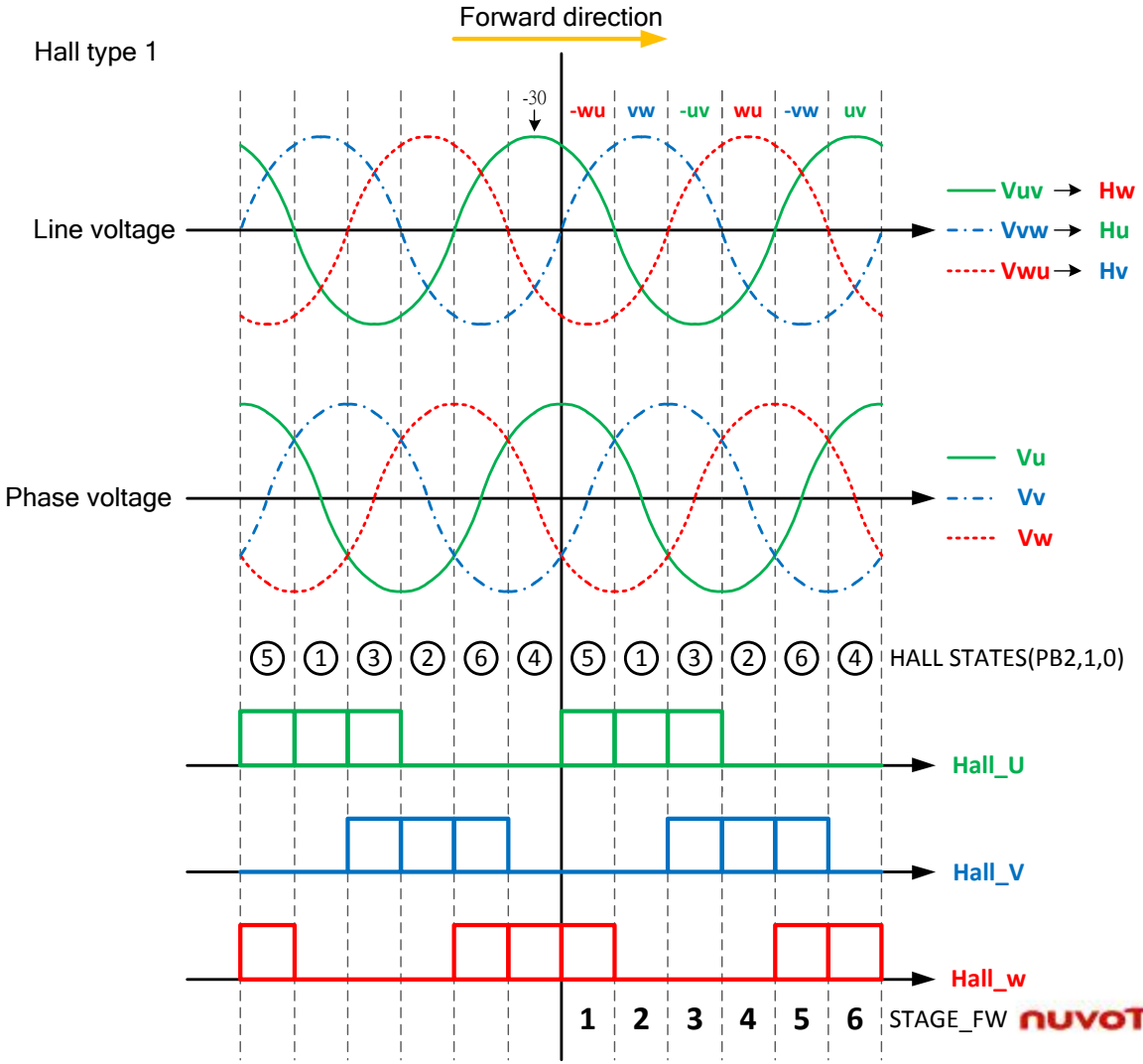
Control mode selection

- ✓ VSP control mode:
  - Rotate VR to make VSP initial voltage < 2.5V before power on.
- ✓ UART control mode:
  - Rotate VR to make VSP initial voltage > 2.5V before power on.

nuvoTon confidential

Hall signal correction

- The correspondence between BEMF & hall signal as bellow
- User can correct with line or phase voltage (example: Type 1)

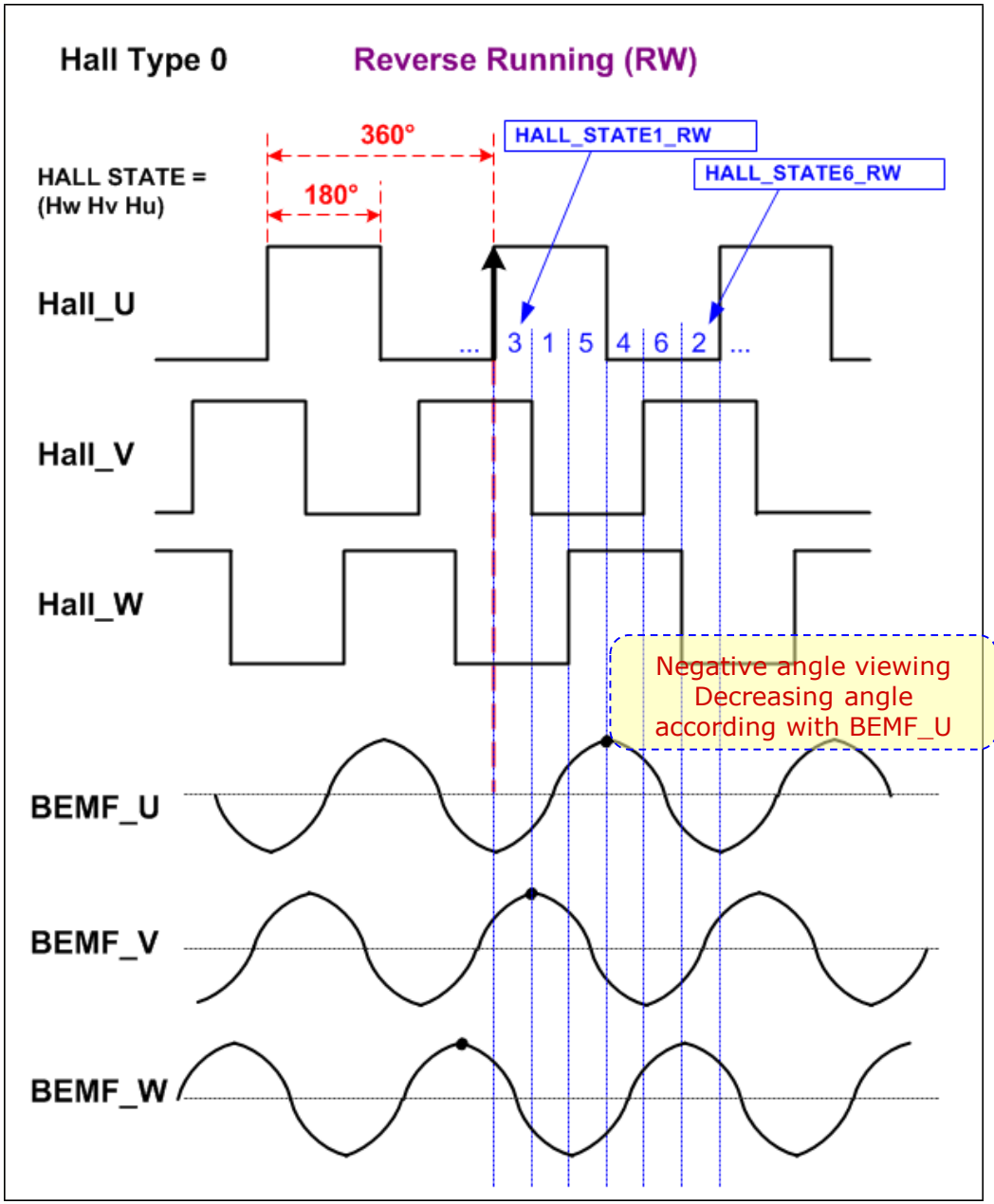


nuvoTon confidential

Hall signal correction

- User can select hall type 0 or type 1 in the example code
- Hall type 0 mean the falling edge of hall signal align phase voltage peak
- Hall type 1 mean the rising edge of hall signal align phase voltage peak

Type0 : Hall and BEMF for RW



Define the 1st hall state right after the rising edge at IC0 as "HALL_STATE1_RW"

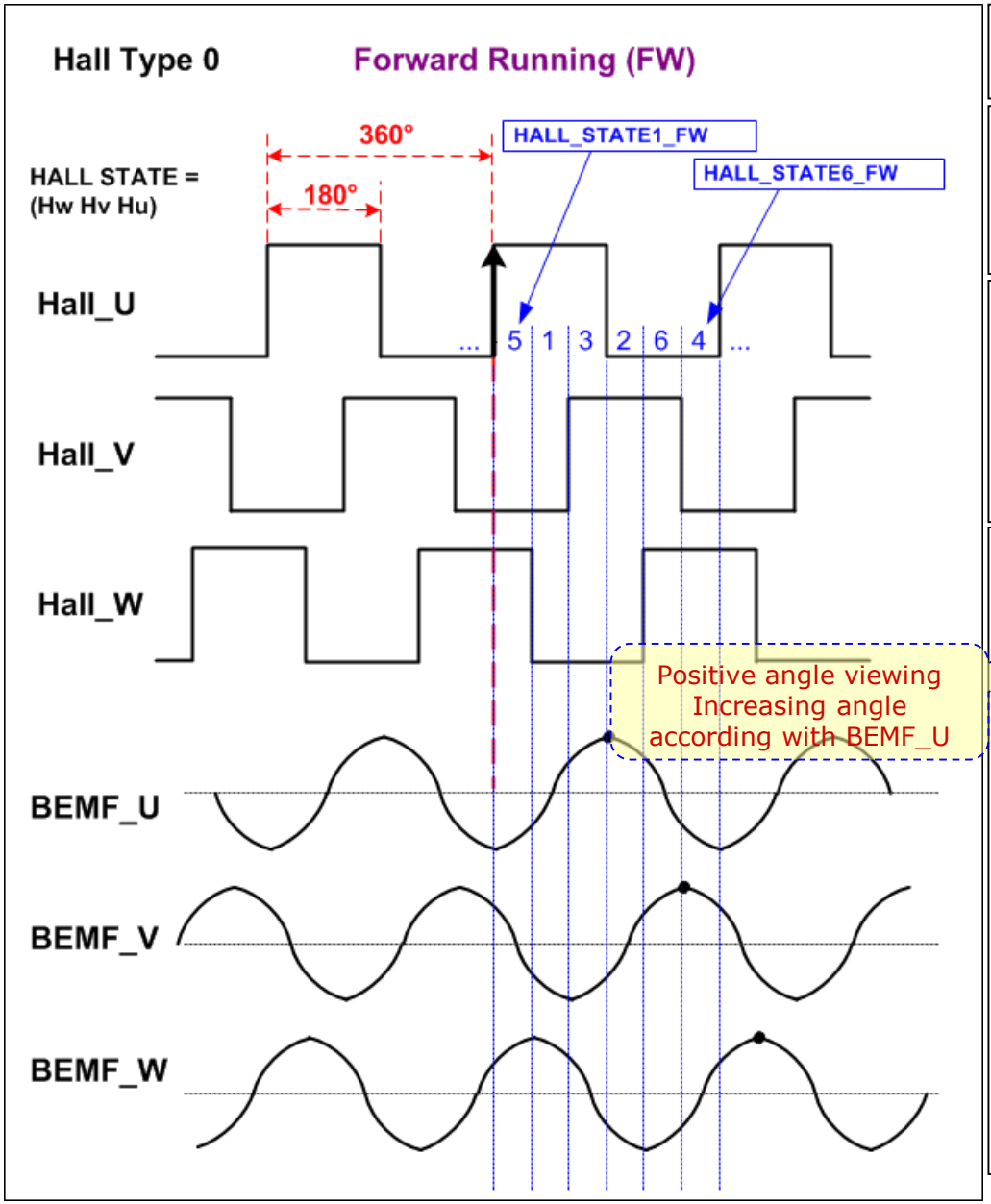
Hall Sensor (W,V,U) =
Input Capture (IC2, IC1, IC0) =
I/O Port (PF2, PF1, PF0) =
3, 1, 5, 4, 6, 2…for CCW

```
#define HALL_STATE1_RW  3
#define HALL_STATE2_RW  1
#define HALL_STATE3_RW  5
#define HALL_STATE4_RW  4
#define HALL_STATE5_RW  6
#define HALL_STATE6_RW  2
```

```
#define HALL_STATE1_R_ANGLE
1024*(-90+0)/360

#define HALL_STATE2_R_ANGLE
1024*(-150+0)/360

#define HALL_STATE3_R_ANGLE
1024*(-210+0)/360

#define HALL_STATE4_R_ANGLE
1024*(-270+0)/360

#define HALL_STATE5_R_ANGLE
1024*(-330+0)/360

#define HALL_STATE6_R_ANGLE
1024*(-30+0)/360
```

nuvoTon confidential

Type0 : Hall and BEMF for FW



Define the 1st hall state right after the rising edge at IC0 as "HALL_STATE1_FW"

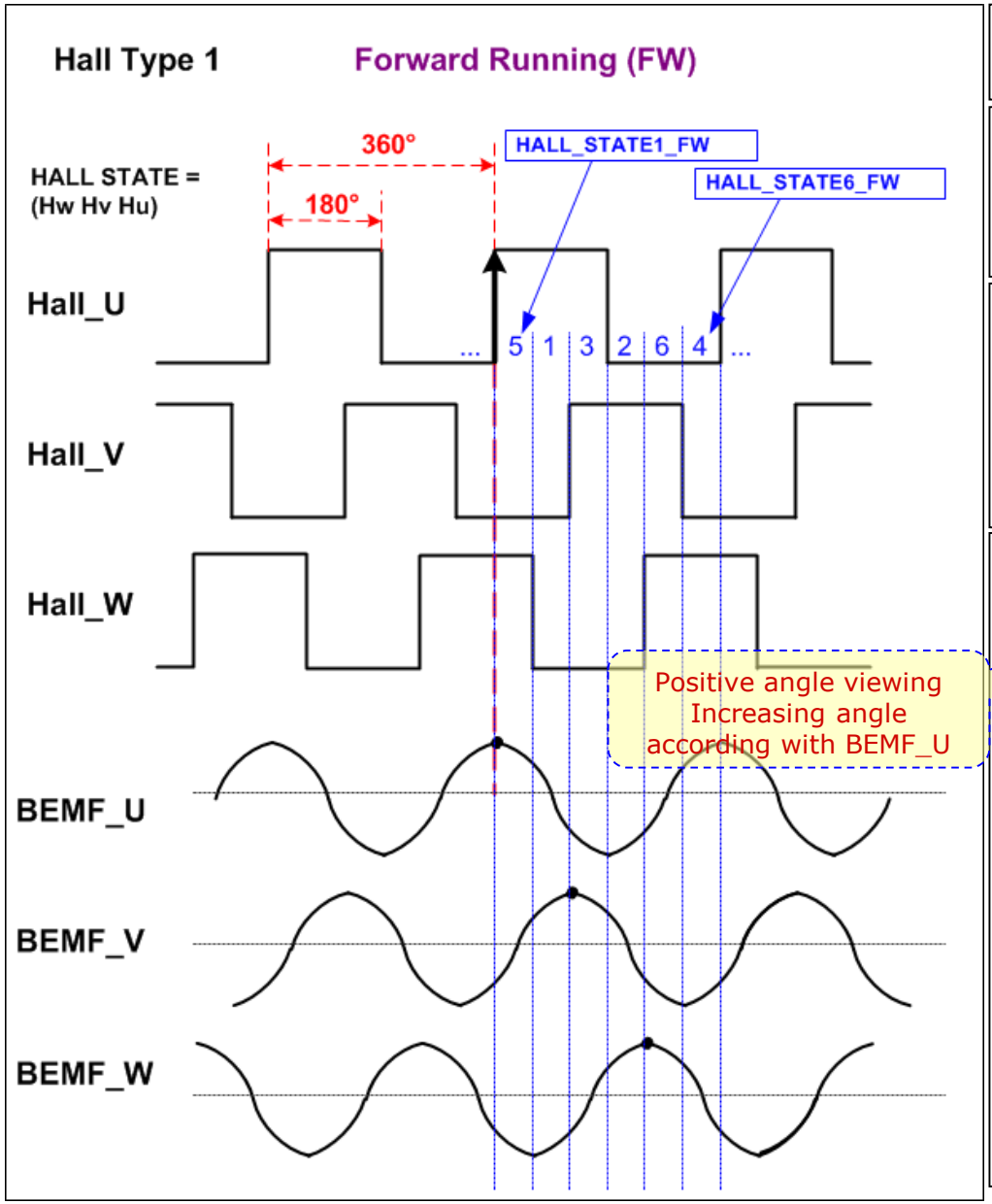
Hall Sensor (W,V,U) =
Input Capture (IC2, IC1, IC0) =
I/O Port (PF2, PF1, PF0) =
5, 1, 3, 2, 6, 4…for FW

```
#define HALL_STATE1_FW  5
#define HALL_STATE2_FW  1
#define HALL_STATE3_FW  3
#define HALL_STATE4_FW  2
#define HALL_STATE5_FW  6
#define HALL_STATE6_FW  4
```

```
#define HALL_STATE1_F_ANGLE
1024*(270+0)/360 + SHIFT_180

#define HALL_STATE2_F_ANGLE
1024*(330+0)/360 + SHIFT_180

#define HALL_STATE3_F_ANGLE
1024*(30+0)/360  + SHIFT_180

#define HALL_STATE4_F_ANGLE
1024*(90+0)/360 + SHIFT_180

#define HALL_STATE5_F_ANGLE
1024*(150+0)/360 + SHIFT_180

#define HALL_STATE6_F_ANGLE
1024*(210+0)/360 + SHIFT_180
```

nuvoTon confidential

Type1 : Hall and BEMF for FW



Define the 1st hall state right after the rising edge at IC0 as "HALL_STATE1_FW"

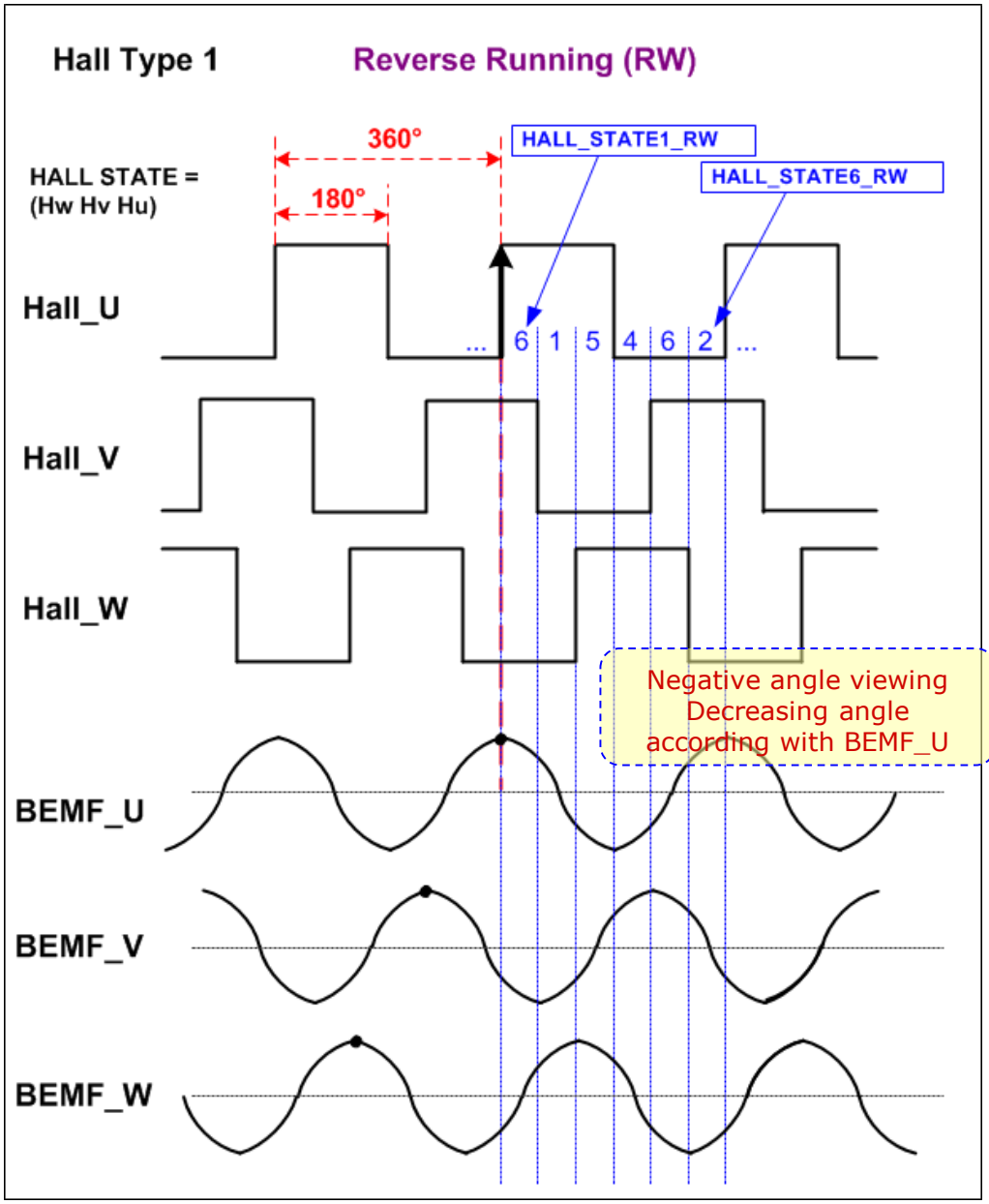
Hall Sensor (W,V,U) =
Input Capture (IC2, IC1, IC0) =
I/O Port (PF2, PF1, PF0) =
5, 1, 3, 2, 6, 4…for FW

```
#define HALL_STATE1_FW  5
#define HALL_STATE2_FW  1
#define HALL_STATE3_FW  3
#define HALL_STATE4_FW  2
#define HALL_STATE5_FW  6
#define HALL_STATE6_FW  4
```

```
#define HALL_STATE1_F_ANGLE
1024*(90+0)/360 + SHIFT_180

#define HALL_STATE2_F_ANGLE
1024*(150+0)/360 + SHIFT_180

#define HALL_STATE3_F_ANGLE
1024*(210+0)/360  + SHIFT_180

#define HALL_STATE4_F_ANGLE
1024*(270+0)/360 + SHIFT_180

#define HALL_STATE5_F_ANGLE
1024*(330+0)/360 + SHIFT_180

#define HALL_STATE6_F_ANGLE
1024*(30+0)/360 + SHIFT_180
```

nuvoTon confidential

Type1 : Hall and BEMF for RW



Define the 1st hall state right after the rising edge at IC0 as "HALL_STATE1_RW"

Hall Sensor (W,V,U) =
Input Capture (IC2, IC1, IC0) =
I/O Port (PF2, PF1, PF0) =
3, 1, 5, 4, 6, 2…for CCW

```
#define HALL_STATE1_RW  3
#define HALL_STATE2_RW  1
#define HALL_STATE3_RW  5
#define HALL_STATE4_RW  4
#define HALL_STATE5_RW  6
#define HALL_STATE6_RW  2
```

```
#define HALL_STATE1_R_ANGLE
1024*(-270+0)/360

#define HALL_STATE2_R_ANGLE
1024*(-330+0)/360

#define HALL_STATE3_R_ANGLE
1024*(-30+0)/360

#define HALL_STATE4_R_ANGLE
1024*(-90+0)/360

#define HALL_STATE5_R_ANGLE
1024*(-150+0)/360

#define HALL_STATE6_R_ANGLE
1024*(-210+0)/360
```

nuvoTon confidential

END